

SHIV NADAR UNIVERSITY CHENNAI

School of Engineering

CS3807 – Deep Learning Laboratory

Experiment – 7

End-to-End Study of Autoencoders , CAE Denoising AE and VAE

Name : Srivatsan S
Register No. : 24011101093
Degree & Branch : B.Tech Artificial Intelligence & Data Science
Class / Section : AIDS – B
Semester : V
Subject Code : CS3807
Subject Name : Deep Learning Laboratory

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

Configuration used in this execution:

Item	Value
Training / validation / test images	9,000 / 1,000 / 2,000 (random subset of MNIST, seed 42)
Pixel scaling	$[0, 255] \rightarrow [0, 1]$
Optimizer, learning rate, batch size	Adam, 10^{-3} , 128
Epochs	20 (FC-AE, CAE, denoising CAE); 30 (VAE)
Reconstruction loss	Binary cross-entropy (VAE: summed over pixels, per image)
Denoiser training noise	Gaussian $\sigma = 0.2$; salt-and-pepper $p = 0.10$ (fresh noise every epoch, clean target)
Epoch-wise MSE/MAE/SSIM	Fixed validation subset (SSIM on the first 300 validation images)
Final metrics	Full 2,000-image test set (MSE, MAE, SSIM via scikit-image)

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

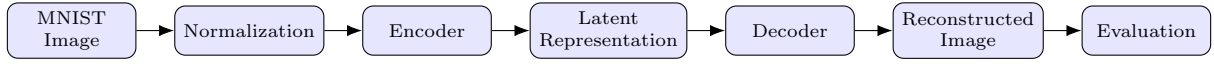
For the denoising model, display:

Original clean image
 Noisy image
 Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

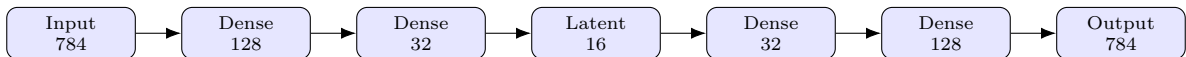
7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



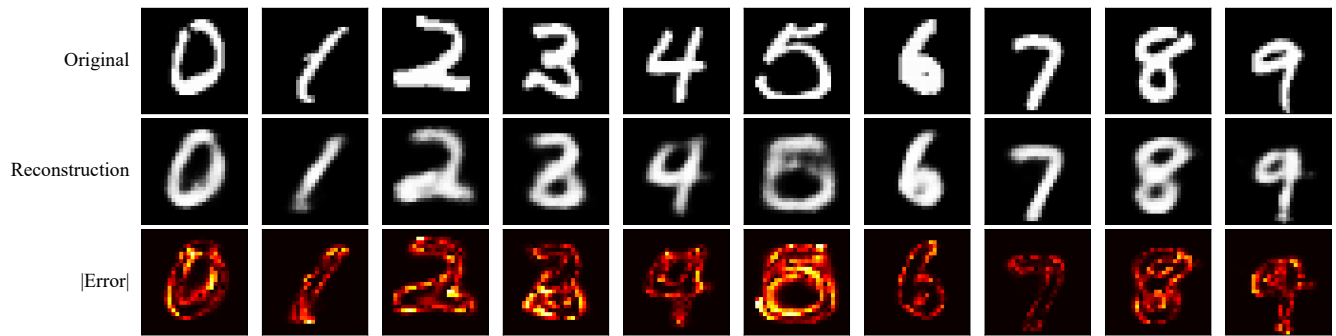
Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

8. Required Analysis of the Basic Autoencoder

Results.



INFERENCE:

Digits like 2, 3 and 5 are smoothed out with errors . Blurring occurs because of the squeezing of the 784 pixels into the 16 latent dimensions . It performs best on simple sparse digits like 1, 7, 0

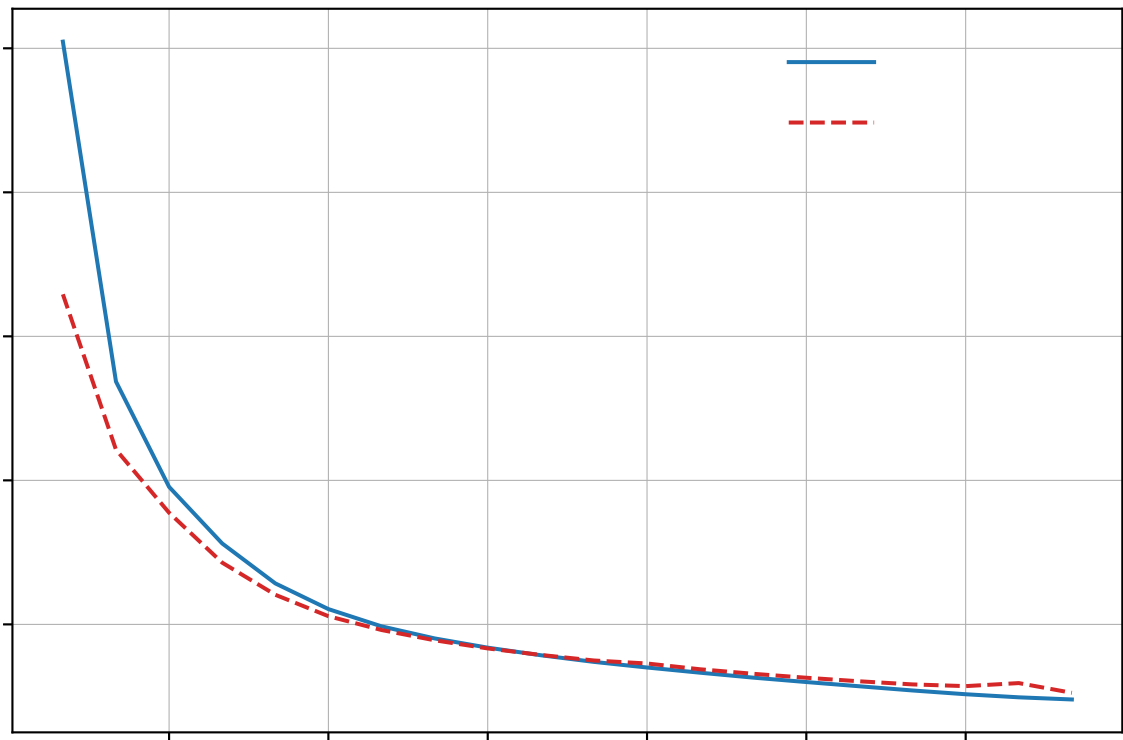


Figure 1: Plot 2: Training and validation binary cross-entropy loss of the fully connected autoencoder over 20 epochs.

INFERENCE: The loss function drops quickly over the 5-6 epochs before settling onto value of 0.124 (training) and 0.126 (validation) .

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

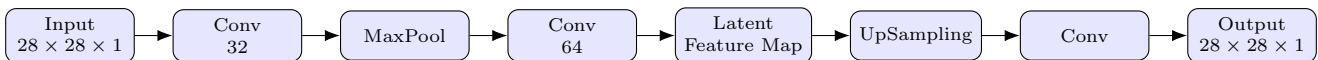
Report:

Metric	Value
Test MSE	0.02068
Test MAE	0.05651
Mean SSIM	0.76262

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```
Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.02068	0.0565	0.7626	211,040
Convolutional AE	0.00269	0.0155	0.9739	74,497

Plot 3: Reconstruction Comparison

Display:

Original → FC-AE Reconstruction → CAE Reconstruction.

Required inference:

Discuss how preserving spatial locality through convolution affects image reconstruction.

Results.

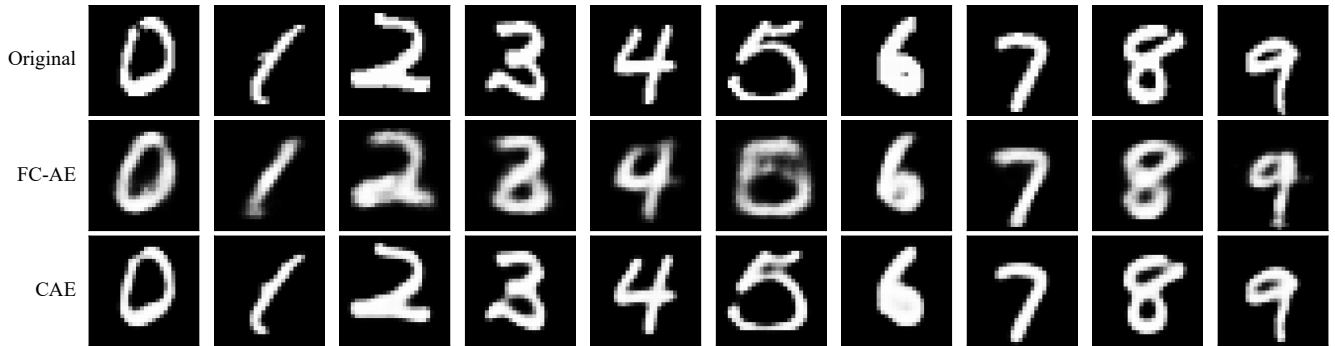


Figure 2: Plot 3: Original test images (top), fully connected AE reconstruction (middle) and convolutional AE reconstruction (bottom).

INFERENCE: Plot 3 The convolutional autoencoder performs better than the fully connected autoencoder with mse of 0.0027 and ssim of 0.974 with only 74,497 parameters compared to the fully connected autoencoder with mse 0.02068 , MAE 0.0155 and SSIM 0.7626 and 211040 parameters .

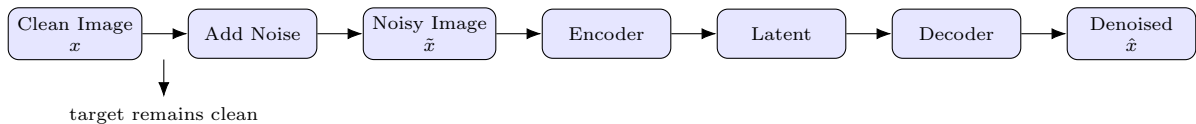
12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image.

The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

14. Required Denoising Plots

Table 1: Gaussian noise: test-set metrics of the denoising CAE (trained at $\sigma = 0.2$) against the clean image. The right block gives the same metrics computed directly on the noisy input, i.e. without denoising.

Noise level σ	Denoised output			Noisy input (no denoising)		
	MSE	MAE	SSIM	MSE	MAE	SSIM
0.1	0.00363	0.0179	0.9613	0.00543	0.0441	0.6971
0.2	0.00455	0.0211	0.9473	0.02122	0.0867	0.6130
0.3	0.00692	0.0292	0.8714	0.04670	0.1282	0.5281

Table 2: Salt-and-pepper noise: test-set metrics of the denoising CAE (trained at $p = 0.10$) against the clean image, with the noisy-input baseline.

Noise level p	Denoised output			Noisy input (no denoising)		
	MSE	MAE	SSIM	MSE	MAE	SSIM
0.05	0.00411	0.0195	0.9571	0.02390	0.0248	0.7249
0.1	0.00505	0.0220	0.9446	0.04799	0.0499	0.5951
0.2	0.00830	0.0302	0.8893	0.09613	0.0998	0.4637



Figure 3: Gaussian noise at $\sigma = 0.1, 0.2, 0.3$: clean images and, for each level, the noisy input and the output of the $\sigma = 0.2$ denoiser.

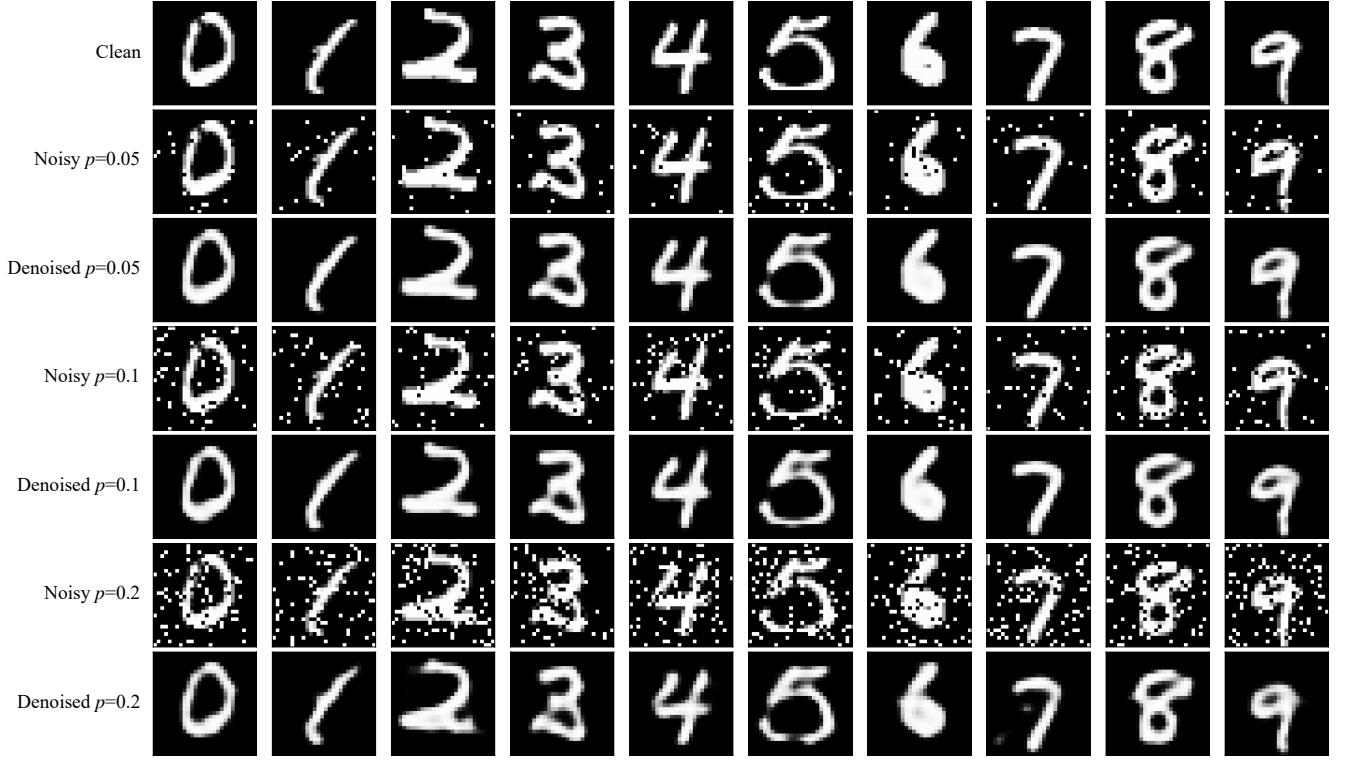


Figure 4: Salt-and-pepper noise at $p = 0.05, 0.10, 0.20$: clean images and, for each level, the noisy input and the output of the $p = 0.10$ denoiser.

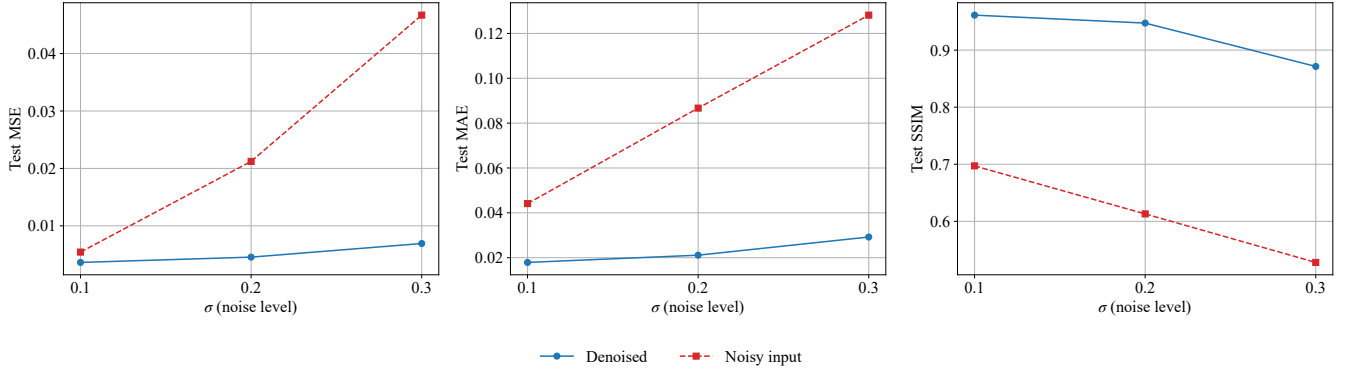


Figure 5: Plot 5 (Gaussian): Test MSE, MAE and SSIM against noise level σ for the denoised output and the noisy input.

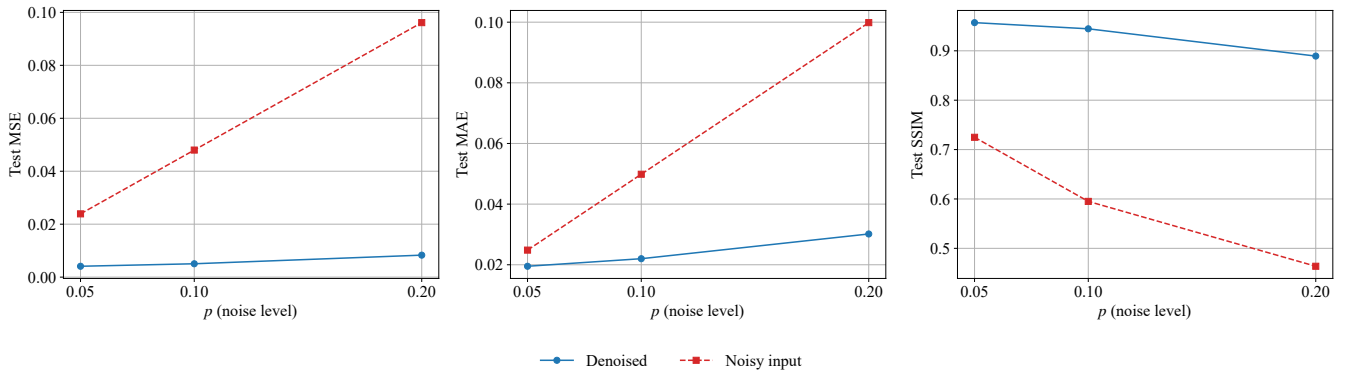


Figure 6: Plot 5 (salt-and-pepper): Test MSE, MAE and SSIM against corruption level p for the denoised output and the noisy input.

The denoising makes CAE learn the digit strokes and clean black backgrounds, mse reduces from 0.0212 to 0.0045 and SSIM raised from 0.613 to 0.947 . The network restores the structural integrity from corrupted inputs

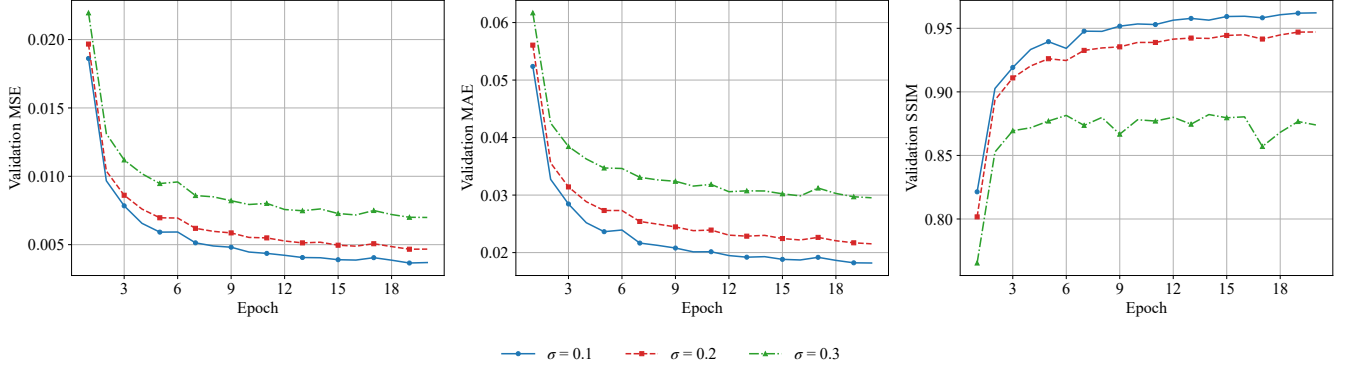


Figure 7: Epoch-wise validation MSE, MAE and SSIM of the Gaussian denoiser at each noise level σ .

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

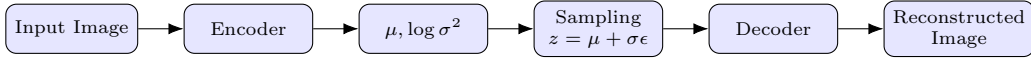
$$q_\phi(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

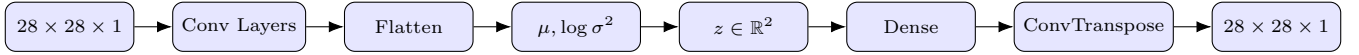
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

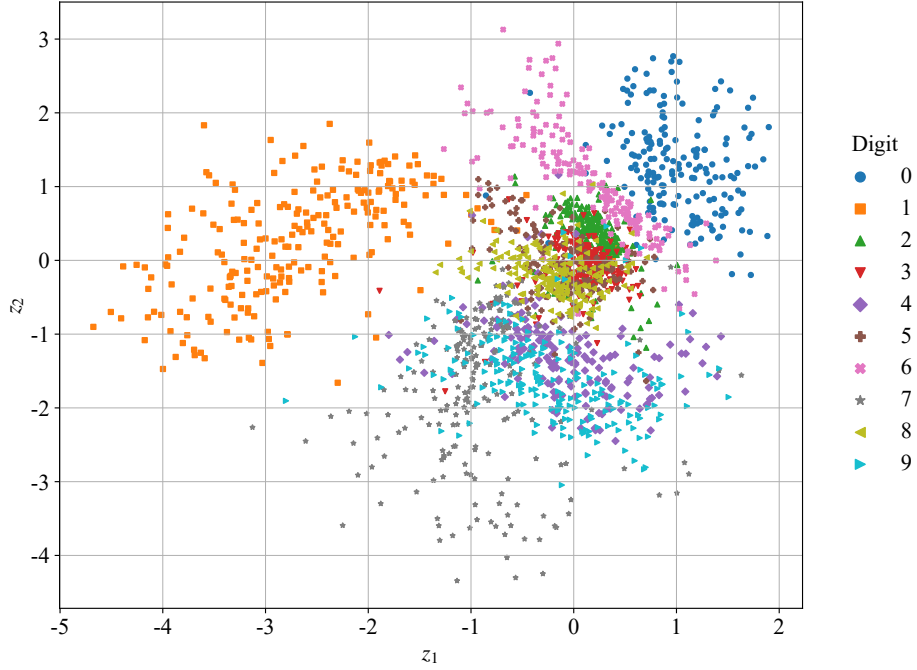


Figure 8: Plot 6: Encoded means (z_1, z_2) of the 2,000 test images, coloured and marked by digit label (labels are not used in training).

The K1 divergence penalty makes the representations into a tight cloud with the class separation being the tradeoff. The VAE organizes latent space with visual similarity and not strict class boundaries which cause the overlapping.

Table 3: Statistics of the encoded test images in the two-dimensional VAE latent space. μ and σ denote the encoder mean and standard deviation; “aggregate std” is $\sqrt{\text{Var}(\mu) + \mathbb{E}[\sigma^2]}$.

Dim.	mean(μ)	std(μ)	mean(σ)	Aggregate std	KL per dim.
z_1	-0.3268	1.1023	0.0544	1.1040	3.1826
z_2	-0.2287	1.1672	0.0637	1.1692	3.0266

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Results.



Figure 9: Plot 7: 25 images generated by decoding latent vectors sampled from $\mathcal{N}(0, I)$ (VAE, $d_z = 2$).

INFERENCE:

The outputs were the handwritten digit shapes which are recognizable however still has blurred and hybrid classes and some classes are missing .

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

Results.

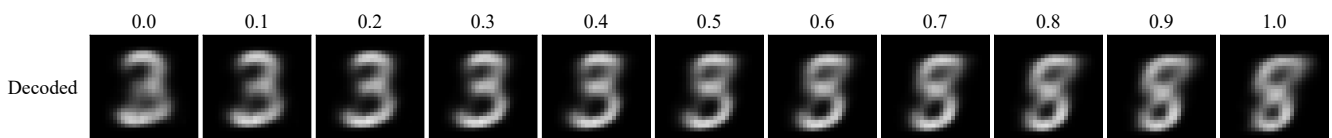


Figure 10: Plot 8: Decoded images along $z(\alpha) = (1 - \alpha)z_A + \alpha z_B$ for $\alpha = 0, 0.1, \dots, 1$; z_A and z_B are the encoded means of a test “3” and a test “8”.

Linear interpolation of encodings from 3 to 8 produces a continuously progressed hybrid transitions with the end resulting in a recognizable digit.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
FC Autoencoder	0.02068	0.0565	0.7626	211,040
Conv. Autoencoder	0.00269	0.0155	0.9739	74,497
Denoising CAE	0.00455	0.0211	0.9473	74,497

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss	147.30
KL Loss	6.21
Total Loss	153.51
Test MSE	0.04217
Test MAE	0.0980
Mean SSIM	0.5363

Notes. The denoising CAE row is the Gaussian denoiser trained and tested at $\sigma = 0.2$ and scored against the clean image. For the VAE, the reconstruction, KL and total losses are per-image values in nats (BCE summed over the 784 pixels, one sample of z); MSE, MAE and SSIM are computed from the reconstruction obtained with $z = \mu$.

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

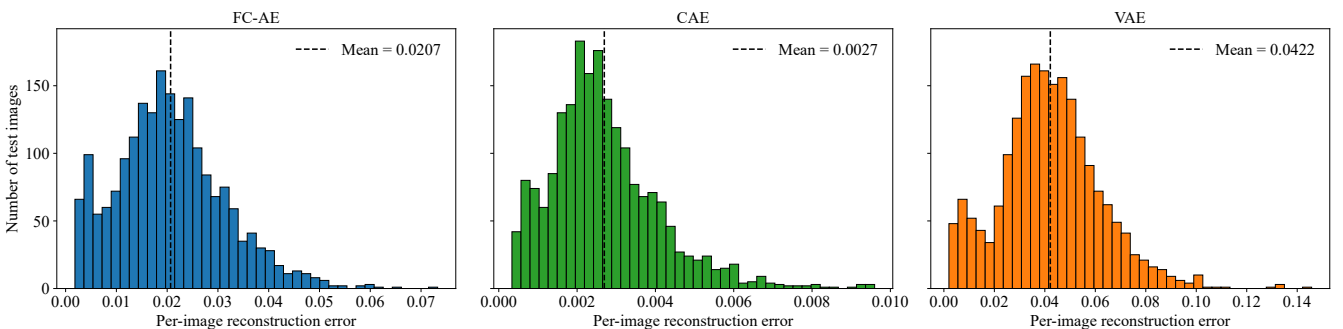


Figure 11: Plot 10: Histogram of the per-image reconstruction error on the test set for the FC-AE, CAE and VAE.

Across the epochs the reconstruction loss drops from 300 to 145 after leveling off while kl divergence rises from 3 to 6.2 nats. The training and validation curves are close enough to conclude that no overfitting had happened

Table 4: Per-image reconstruction error e_i (mean squared error per image) over the 2,000 test images. P95 is the 95th percentile.

Model	Mean	Median	Std	P95	Max
FC-AE	0.02068	0.01987	0.01062	0.03959	0.07327
CAE	0.00269	0.00249	0.00139	0.00533	0.00960
VAE	0.04217	0.04114	0.01970	0.07503	0.14572

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

Display:

Table 5: The five test images with the largest reconstruction error for each model (rank 1 = largest error). The originals and reconstructions are shown in Figures 12 and 13.

Sample	Fully connected AE			Convolutional AE		
	Test index	Digit	MSE	Test index	Digit	MSE
1	1	9	5.00000	1	892	6.00000
2	2	660	6.00000	2	1317	0.00000
3	3	327	6.00000	3	152	8.00000
4	4	1903	2.00000	4	682	8.00000
5	5	115	5.00000	5	1980	8.00000

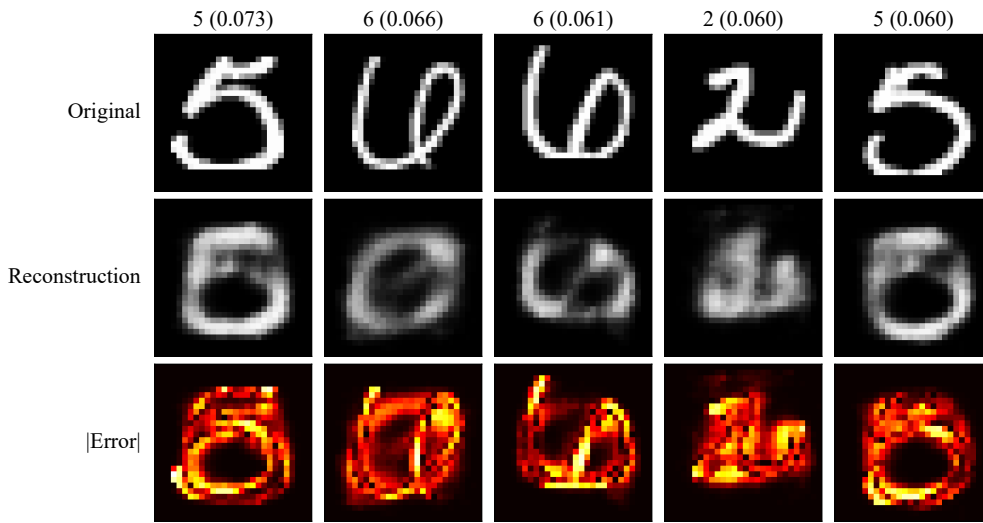


Figure 12: Five highest-error test images for the FC-AE: original (top), reconstruction (middle) and absolute error (bottom). Column titles give the true digit and the per-image MSE.

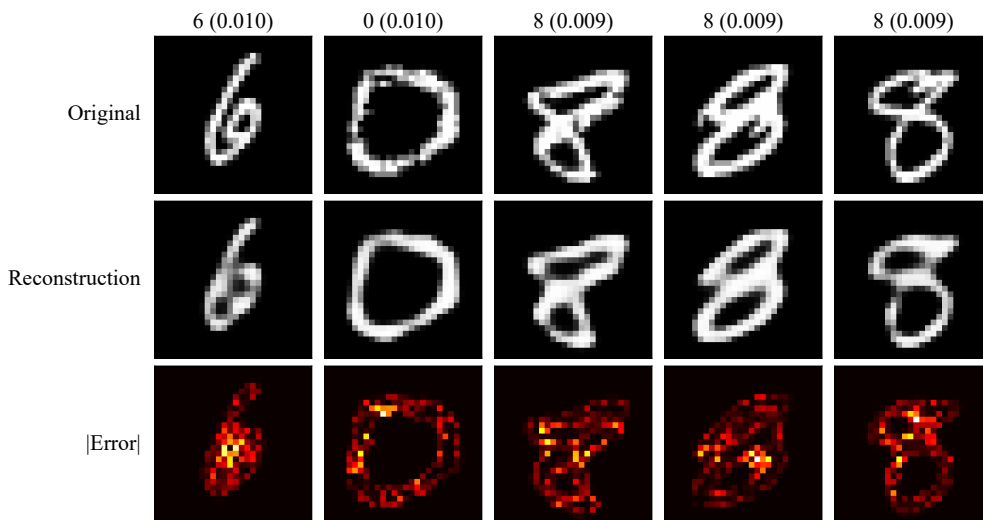


Figure 13: Five highest-error test images for the CAE: original (top), reconstruction (middle) and absolute error (bottom).

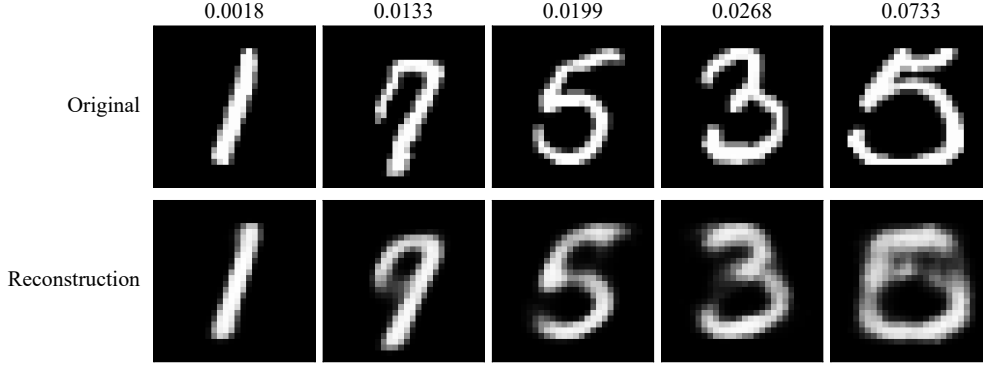


Figure 14: FC-AE reconstructions at the 0th, 25th, 50th, 75th and 100th percentile of per-image error (column titles give the MSE).

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time (s)
FC Autoencoder	0.02068	0.0565	0.7626	211,040	14.8
Conv. Autoencoder	0.00269	0.0155	0.9739	74,497	23.6
Denoising CAE	0.00455	0.0211	0.9473	74,497	36.7
VAE	0.04217	0.0980	0.5363	485,957	39.8

The values must be obtained from the student's own execution.

Notes. Time is the wall-clock training time on the Colab GPU. The VAE was trained for 30 epochs and the other models for 20. The Denoising CAE row is the Gaussian $\sigma = 0.2$ model tested at $\sigma = 0.2$; the salt-and-pepper results are in Table 2. VAE MSE, MAE and SSIM are computed from the $z = \mu$ reconstruction.

27. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Latent Dimension	MSE	MAE	SSIM	Parameters
2	0.04728	0.1097	0.4761	210,130
8	0.02499	0.0650	0.7208	210,520
16	0.02068	0.0565	0.7626	211,040
32	0.01832	0.0515	0.7876	212,080

Plot 10: Latent Dimension vs. Reconstruction MSE

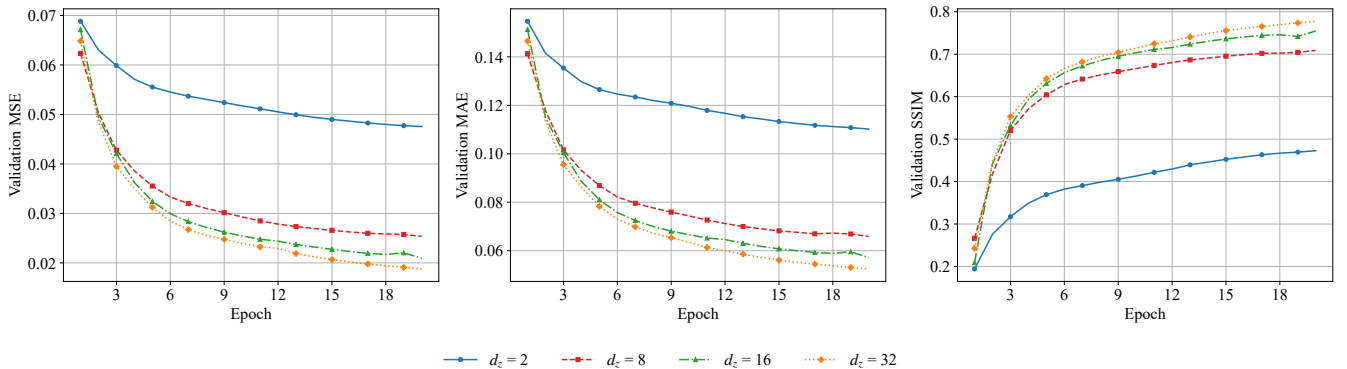


Figure 15: Epoch-wise validation MSE, MAE and SSIM of the fully connected autoencoder for each latent dimension.

The images are difficult because of the relationship between reconstruction error and visual quality. Low error samples consist of low strokes because pixel error scaled with visual distortions. High errors consist of cursive complex digits causing blurred averages to be taken

28. Discussion Questions

1. What is an autoencoder? An autoencoder is a neural network architecture designed to learn efficient, low-dimensional representations of unlabelled input data. It works by compressing the input into a smaller bottleneck code and then reconstructing the original input as closely as possible from that code.
2. What is the purpose of the encoder? The encoder's job is to compress the high-dimensional input into a compact, low-dimensional vector. It extracts essential underlying features and patterns while discarding redundant details or noise.
3. What is the purpose of the decoder? The decoder takes the low-dimensional latent vector generated by the encoder and attempts to reconstruct the original high-dimensional input from it.
4. What is a latent representation? A latent representation (or bottleneck code) is a compressed, lower-dimensional vector space where the network encodes the most critical information and semantic features of the input data.
5. Why is the latent representation usually smaller than the input? Squeezing the latent space forces the network to learn meaningful data compressions instead of simply memorizing the input through a direct identity mapping.
6. What happens when the latent dimension is too small? If the latent space is too small, the network suffers from an information bottleneck. It loses critical fine details, causing reconstructed outputs to become blurry, distorted, or structurally incomplete.
7. What happens when the latent dimension is increased? Increasing the latent dimension expands the model's capacity to preserve detail and lowers reconstruction error. However, making it too large risks losing compression benefits or letting the model memorize the training data.
8. Why can a convolutional autoencoder preserve spatial information more effectively? Convolutional autoencoders preserve spatial information by maintaining 2D feature maps and utilizing shared local weights, allowing them to capture spatial hierarchies and geometric edges much better than dense, fully connected layers.
9. Differentiate an autoencoder and a denoising autoencoder. A standard autoencoder is trained to reconstruct clean inputs, whereas a denoising autoencoder takes corrupted or noisy inputs and learns to recover the clean, uncorrupted version.
10. Why is the clean image used as the target in denoising? Using the clean image as the target teaches the model the structural patterns of valid data, allowing it to ignore random pixel-level corruptions as non-essential noise.
11. What happens when the noise level is increased? Higher noise levels strip away more original pixel information, making reconstruction harder and causing higher error metrics along with smoother or blurrier outputs.
12. Why are MSE, MAE and SSIM useful for image reconstruction? MSE measures average squared pixel errors, MAE measures absolute distance to penalize outliers less heavily, and SSIM evaluates structural similarity, brightness, and contrast as perceived by human vision.
13. What is a Variational Autoencoder? A Variational Autoencoder (VAE) is a probabilistic generative model that maps inputs to a probability distribution over the latent space rather than a single fixed point.
14. How does a VAE differ from a conventional autoencoder? Standard autoencoders map inputs to fixed, deterministic vectors in latent space, whereas VAEs map inputs to continuous distributions defined by a mean and a variance vector.
15. Why does a VAE learn a distribution instead of a single latent vector? Learning a continuous probability distribution ensures the latent space stays smooth and complete, preventing isolated clusters and allowing meaningful generation and interpolation.
16. What is the reparameterization trick? The reparameterization trick makes sampling differentiable by expressing the latent code as $z = \mu + \sigma * \epsilon$, shifting random variation onto an independent random variable ϵ .
17. Why is ϵ sampled from a standard normal distribution? ϵ is sampled from a standard normal distribution to provide random noise while keeping backpropagation gradients clear for the mean and variance parameters.
18. What is the role of KL divergence in the VAE loss? The KL divergence acts as a regularization term, forcing the learned latent distributions to stay close to a standard normal prior and keeping the latent space smooth and continuous.
19. What is the prior distribution used in the VAE? The prior distribution in a VAE is typically a standard normal distribution $N(0, I)$, which acts as a standard reference point for both regularizing and generating samples.
20. Why is a two-dimensional latent space useful for visualization? A two-dimensional latent space lets you easily plot and visualize how classes, features, and continuous boundaries are arranged across the data space.
21. What does latent-space interpolation demonstrate? Latent-space interpolation shows that the latent manifold is smooth and continuous, proving the network learns meaningful structural shapes rather than just memorizing training examples.
22. Why can a VAE generate new images? VAEs can generate new images by decoding random samples drawn straight from the learned prior distribution $N(0, I)$.

23. Why might some VAE-generated images be unclear? Generated images can appear blurry due to information loss in small latent dimensions, as well as the MSE or BCE loss functions penalizing sharp edges in slightly wrong positions.
24. What is the difference between reconstruction and generation? Reconstruction involves mapping an existing image to latent space and rebuilding it, whereas generation creates a brand-new sample by decoding a random point sampled directly from the latent prior.
25. Why should test images remain unseen during training? Keeping test images unseen during training confirms that the autoencoder can generalize its learned features to new data rather than just memorizing specific training images.

29. Additional Exercises

Results of the Additional Exercises

Exercise 1: CAE with latent dimensions 4, 8, 16 and 32

A dense bottleneck (Flatten $\rightarrow$ Dense(d_z) $\rightarrow$ Dense $\rightarrow$ Reshape) is inserted between the convolutional encoder and decoder. The last row is the main CAE of Section 10 with a $7 \times 7 \times 64$ feature-map latent, given for reference.

Table 6: Convolutional autoencoder with a d_z -dimensional dense bottleneck (test set).

Latent dim.	MSE	MAE	SSIM	Parameters	Time (s)
4	0.03323	0.0802	0.6574	65,797	24.0
8	0.02017	0.0551	0.7796	90,889	22.2
16	0.01043	0.0339	0.8910	141,073	22.1
32	0.00664	0.0253	0.9307	241,441	23.2
$7 \times 7 \times 64$ map (3136)	0.00269	0.0155	0.9739	74,497	23.6

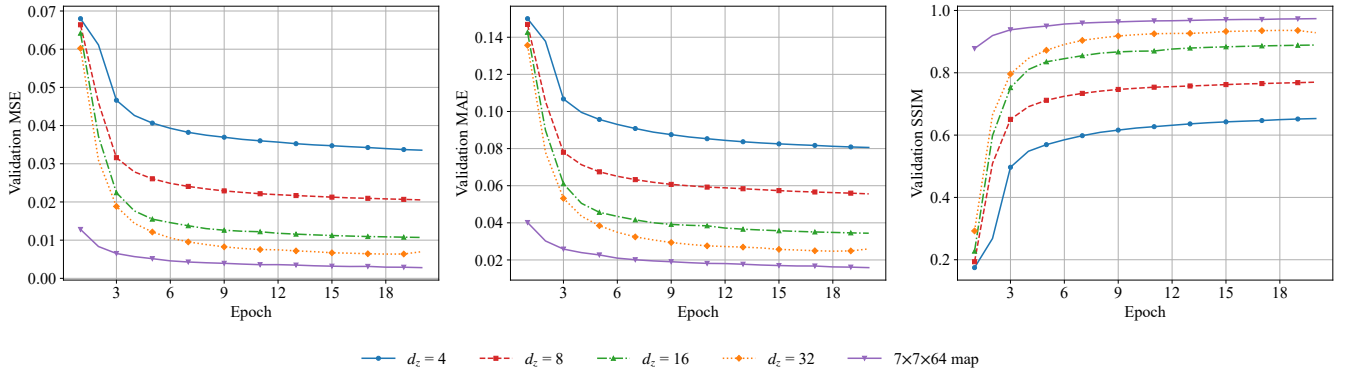


Figure 16: Exercise 1: Epoch-wise validation MSE, MAE and SSIM of the CAE for each bottleneck size, with the main CAE as reference.

Exercise 2: Gaussian versus salt-and-pepper noise

The same denoising CAE architecture is trained once per noise type ($\sigma = 0.2$ and $p = 0.10$) and each model is tested on both noise types at all levels.

Table 7: Cross-evaluation of the two denoisers on the test set (metrics against the clean image).

Trained on	Tested on	Level	MSE	MAE	SSIM
Gaussian	Gaussian	0.1	0.00363	0.0179	0.9613
		0.2	0.00455	0.0211	0.9473
		0.3	0.00692	0.0292	0.8714
	Salt-and-pepper	0.05	0.00467	0.0208	0.9344
		0.1	0.00668	0.0264	0.8851
		0.2	0.01285	0.0426	0.7600
Salt-and-pepper	Gaussian	0.1	0.00677	0.0286	0.8940
		0.2	0.01200	0.0455	0.7298
		0.3	0.01938	0.0674	0.6280
	Salt-and-pepper	0.05	0.00411	0.0195	0.9571
		0.1	0.00505	0.0220	0.9446
		0.2	0.00830	0.0302	0.8893

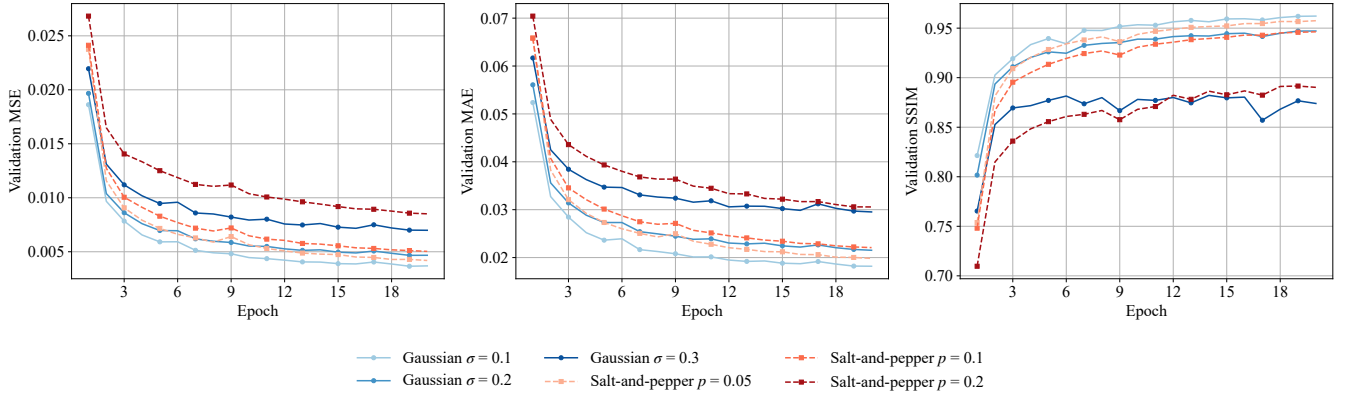


Figure 17: Exercise 2: Epoch-wise validation MSE, MAE and SSIM of the two denoisers at every noise level (blue: Gaussian σ , red: salt-and-pepper p).

Exercise 3: Denoising CAE trained at different noise levels

Denoisers are trained at $\sigma = 0.1, 0.2$ (main model) and 0.3 and every model is tested at all three σ .

Table 8: Test-set metrics for each training noise level σ_{train} and test noise level σ_{test} .

σ_{train}	σ_{test}	MSE	MAE	SSIM
0.1	0.1	0.00320	0.0173	0.9654
	0.2	0.00499	0.0244	0.9015
	0.3	0.00956	0.0419	0.7302
0.2	0.1	0.00363	0.0179	0.9613
	0.2	0.00455	0.0211	0.9473
	0.3	0.00692	0.0292	0.8714
0.3	0.1	0.00432	0.0194	0.9539
	0.2	0.00497	0.0216	0.9447
	0.3	0.00652	0.0262	0.9225

Table 9: Test SSIM matrix (rows: training σ , columns: test σ).

$\sigma_{\text{train}} \backslash \sigma_{\text{test}}$	0.1	0.2	0.3
0.1	0.9654	0.9015	0.7302
0.2	0.9613	0.9473	0.8714
0.3	0.9539	0.9447	0.9225

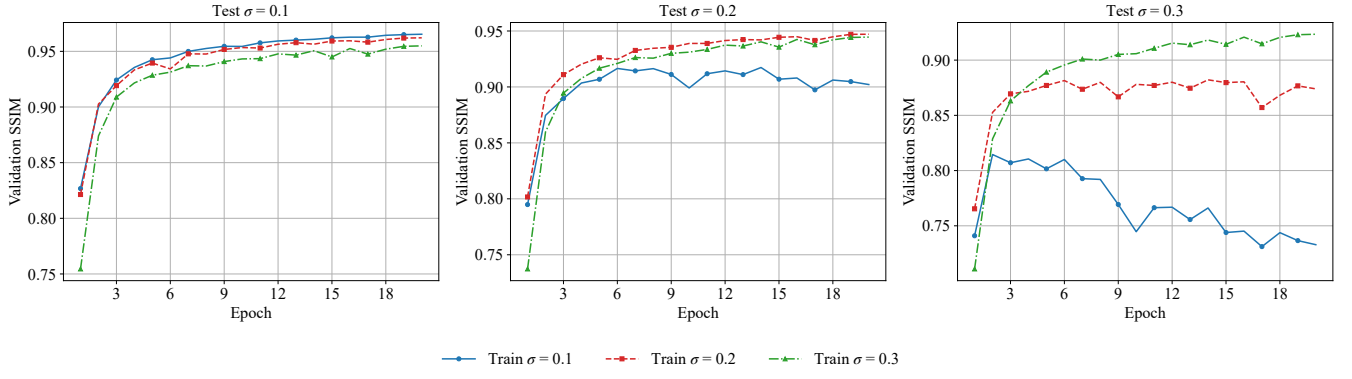


Figure 18: Exercise 3: Epoch-wise validation SSIM for each training σ , one panel per test σ .

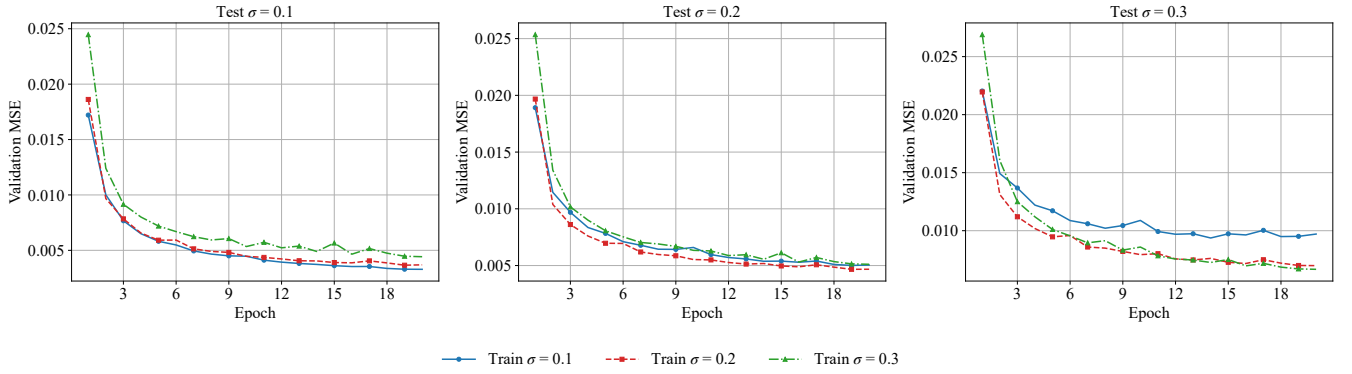


Figure 19: Exercise 3: Epoch-wise validation MSE for each training σ , one panel per test σ .

Exercise 4: Transposed-convolution decoder

Table 10: CAE with an UpSampling2D + Conv2D decoder versus a Conv2DTranspose decoder (test set).

Decoder	MSE	MAE	SSIM	Parameters	Time (s)
UpSampling2D + Conv2D	0.00269	0.0155	0.9739	74,497	23.6
Conv2DTranspose	0.00233	0.0144	0.9765	74,497	22.3

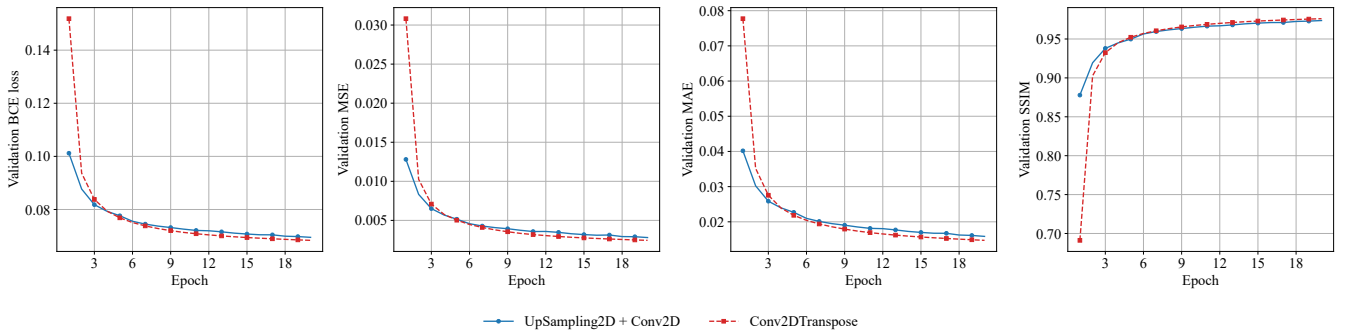


Figure 20: Exercise 4: Epoch-wise validation BCE loss, MSE, MAE and SSIM for the two decoder types.

Exercise 5: VAE with latent dimension 2 and 8

Table 11: VAE with $d_z = 2$ and $d_z = 8$ (test set). Losses are per-image values in nats.

VAE	Rec. loss	KL loss	Total loss	MSE	MAE	SSIM	Params	Time (s)
$d_z = 2$	147.30	6.21	153.51	0.04217	0.0980	0.5363	485,957	39.8
$d_z = 8$	98.06	16.35	114.41	0.01913	0.0523	0.7971	506,321	28.9

Table 12: Per-dimension statistics of the encoded test images for the $d_z = 8$ VAE. The two leading principal components of the encoded means explain 20.5% and 15.3% of the variance.

Dim.	std(μ)	mean(σ)	KL per dim.
z_1	0.9268	0.0904	2.3792
z_2	1.0355	0.1064	2.3072
z_3	0.8058	0.3168	1.0607
z_4	1.0566	0.1122	2.2633
z_5	0.9312	0.1331	1.9893
z_6	0.9944	0.0996	2.3288
z_7	1.0582	0.1052	2.3400
z_8	0.9126	0.1790	1.6808

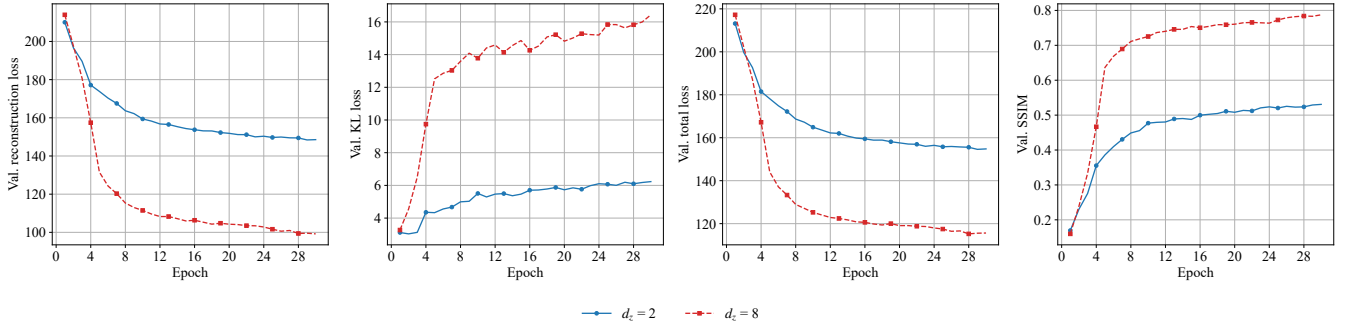


Figure 21: Exercise 5: Epoch-wise validation reconstruction loss, KL loss, total loss and SSIM for $d_z = 2$ and $d_z = 8$.

Exercise 6: Effect of the KL-loss weight

The objective is $\mathcal{L} = \mathcal{L}_{rec} + \beta \mathcal{L}_{KL}$ with $\beta \in \{0.5, 1, 2, 5\}$ and $d_z = 2$. The total loss includes the factor β and is therefore not comparable across rows. The $\beta = 1$ row re-evaluates the main VAE with a new draw of ϵ , so its losses differ slightly from the VAE table of Section 21.

Table 13: Effect of the KL weight β on the $d_z = 2$ VAE (test set).

β	Rec. loss	KL loss	Total loss	MSE	MAE	SSIM	Params	Time (s)
0.5	147.45	7.03	150.97	0.04235	0.0985	0.5363	485,957	34.0
1	147.22	6.21	153.43	0.04217	0.0980	0.5363	485,957	39.8
2	151.30	4.91	161.11	0.04347	0.1021	0.5147	485,957	29.0
5	156.04	3.42	173.13	0.04403	0.1047	0.4987	485,957	28.4

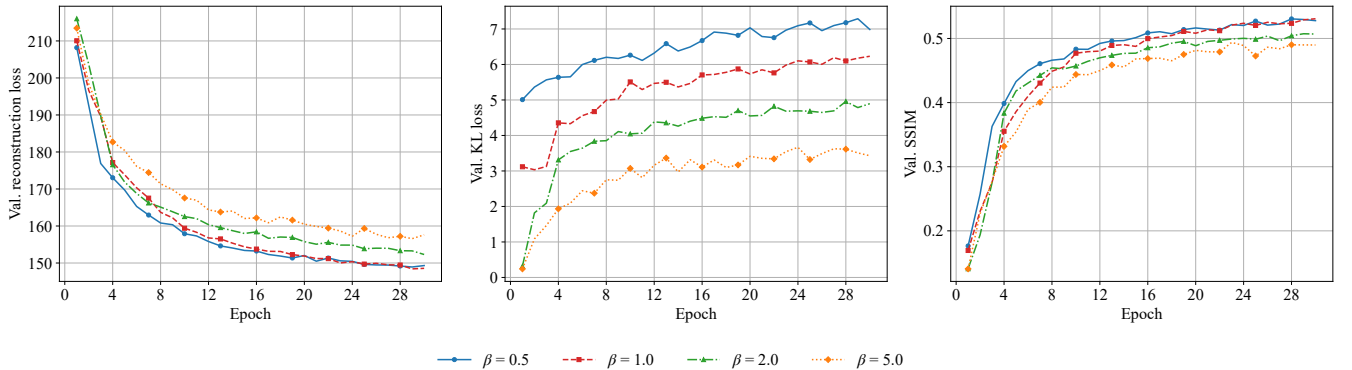


Figure 22: Exercise 6: Epoch-wise validation reconstruction loss, KL loss and SSIM for each β .

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image $\rightarrow$ Encoder $\rightarrow$ Latent Representation $\rightarrow$ Decoder $\rightarrow$ Reconstructed Image

They should also demonstrate denoising:

Clean Image $\rightarrow$ Noise $\rightarrow$ Denoising Autoencoder $\rightarrow$ Recovered Image

and variational generation:

Image $\rightarrow (\mu, \sigma) \rightarrow z \rightarrow$ Decoder $\rightarrow$ Generated/Reconstructed Image

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

SOURCE CODE

<https://github.com/sriwhatsnext/dllab/tree/main/Week07>

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>